STAGE 5 REPORT

Project closure

Ko.tools is a reservation management web application designed for cultural structures.

Developed as a solo end-of-year project at Holberton School, it enables authenticated users to create, view, and update reservations through a dynamic interface connected to a REST API built with Flask.

1. Project summary

MVP functionalities:

- Reservation creation form with dynamic rendering
- Monthly planning view displaying reservation cards with month navigation
- Reservation detail view and update with partially editable fields
- Admin access to every user's planning and reservations

Initial Objectives vs Outcomes:

By extending the timeline to accommodate MySQL and admin feature implementation, the project achieved all MVP goals despite the initial delay.
The following points summarize the achievement rate against the objectives defined in the Project Charter.

- Core reservation workflow — create, read, update: fully delivered
- JWT authentication and role-based access control (API): fully delivered
- Admin access to users plannings and reservations: implemented
- MySQL database: fully implemented

100% of the features has been delivered

Key metrics :

- 9 entities
- 15 API routes tested through 34 tests
- ~ 125 files across back-end, front-end, tests
- 49 catalog objects seeded into the database with a env.json file to import them as environment variables in Postman

2. Lessons learned:

What Went Well

The ER diagram designed in stage 3 proved solid — no data model rework was needed during development, allowing full focus on business logic.

Back-end architecture was well separated: repositories handle data access, services enforce business rules, and the API layer handles requests and responses.

Going directly to ORM models instead of in-memory repositories was an efficient decision that helped recover the initial delay.

The seed script and Postman environment variable export saved significant time during API testing.

Challenges Encountered

Date handling caused repeated bugs: format mismatches between JavaScript (toISOString timezone shift), query parameters (strings), and SQLAlchemy (date objects).

The audience types model lacked fields for proper front-end rendering, requiring a late model rebuild that introduced new bugs and consumed significant time.

Plain JavaScript front-end became hard to maintain beyond 300+ lines per file, requiring mid-development refactoring.

Manager features were implemented on the back-end but had no front-end continuity, resulting in partially wasted development time.

Improvements for Future Projects

Identify unfamiliar data types and their formats early in the design phase — especially types that travel across multiple layers of the stack such as dates.

Design data models with front-end rendering needs in mind: think about what data is needed to display before writing the model.

Use a front-end framework (React, Vue) to keep rendering logic concise, readable, and maintainable.

Adopt an Agile approach: delivering fully functional vertical slices avoids building back-end features that never reach the user.

Automate API tests with assertions instead of relying solely on exploratory Postman testing.